

МИНОБРНАУКИ РОССИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ

**«САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ ПЕТРА ВЕЛИКОГО»**

Институт Компьютерных наук и кибербезопасности

Направление 02.03.01 Математика и компьютерные науки

Высшая школа технологий искусственного интеллекта

**Лабораторная работа 1 «Реализация
двумерного клеточного автомата» по
дисциплине «Теория алгоритмов»**

Вариант 39

Обучающийся: _____

Санько В. В.

Руководитель: _____

Востров А. В.

«_____» _____ 20__ г.

Санкт-Петербург, 2025

Содержание

Введение	3
1 Математическое описание	4
1.1 Клеточный автомат	4
1.2 Окрестность фон Неймана	5
1.3 Функция перехода	5
1.4 Граничные условия	5
2 Особенности реализации	7
2.1 class NeumannCellularAutomaton	7
2.2 Функция main()	8
2.3 Генерация правил из номера (_generate_rules_from_id)	10
2.4 Получение соседей по окрестности фон Неймана (get_neumann_neighbors)	10
2.5 Кодирование состояния окрестности (get_neighborhood_state)	11
2.6 Обновление состояния автомата (update)	11
2.7 Установка случайного начального состояния (set_random_initial_state)	11
2.8 Установка начального состояния (set_manual_initial_state)	12
2.9 Консольная симуляция (run_console_simulation)	14
2.10 Вывод сетки в консоль (print_grid)	14
2.11 Визуализация симуляции (visualize_simulation)	15
2.12 Анализ поведения (analyze_behavior)	16
2.13 Построение графиков анализа (plot_behavior_analysis)	17
3 Результаты работы программы	19
4 Анализ клеточного автомата	22
Заключение	25
Список материалов	26

Введение

В рамках данной лабораторной работы необходимо реализовать двумерный клеточный автомат с окрестностью фон Неймана в соответствии с заданным вариантом.

В соответствии с вариантом было использовано число 59350005.

Необходимо дать возможность пользователю самостоятельно определять ширину поля и количество итераций. Также необходимо предусмотреть возможность ввода различных начальных условий (как вручную, так и случайным образом) по выбору пользователя.

Реализовать программу можно как в консоли, так и в графическом интерфейсе.

В качестве граничных условий реализовать тороидальные граничные условия.

В результате необходимо проанализировать поведение клеточного автомата в отчёте, включая паттерны, сходимость и другие характеристики.

1 Математическое описание

1.1 Клеточный автомат

Клеточный автомат — дискретная динамическая система, определяемая кортежем:

$$CA = (L, S, N, f), \text{ где:}$$

- L — регулярная решетка размерности d
- S — конечное множество состояний
- N — окрестность клетки
- $f : S^n \rightarrow S$ — локальная функция перехода

Двумерный клеточный автомат можно определить как множество конечных автоматов на плоскости, помеченных целочисленными координатами $(i, j) \in [0, W - 1] \times [0, H - 1]$, каждый из которых может находиться в одном из состояний $S = \{0, 1\}$.

Функция, аргументами которой являются состояния соседей клетки, вычисляющая новое состояние для клетки определяется следующим образом:

$$\phi(A, B, C, D, E).$$

Новое состояние клетки:

$$new_s = \phi(A, B, C, D, E).$$

Для каждой клетки определено множество клеток, называемых окрестностью. Устанавливаются правила перехода клеток из одного состояния в другое. Обычно правила перехода одинаковы для всех клеток. В данной работе используется окрестность фон Неймана Рис. 1.

Один шаг автомата подразумевает обход всех клеток и на основе данных о текущем состоянии клетки и её окрестности определение нового состояния клетки, которое будет у неё при следующем шаге.

Эволюция клеточного автомата во времени описывается уравнением:

$$s(t + 1, x) = f(N(s(t, x))), \text{ где:}$$

- $s(t, x)$ — состояние клетки x в момент времени t
- $N(s(t, x))$ — состояния клеток в окрестности фон Неймана
- f — функция перехода, определяющая новое состояние клетки

1.2 Окрестность фон Неймана

Для двумерного случая окрестность фон Неймана радиуса $r = 1$ включает 5 клеток. Для центральной клетки с координатами $(i, j) \in [0, W - 1] \times [0, H - 1]$ окрестность определяется как:

$$N(i, j) = \{(i, j), (i - 1, j), (i + 1, j), (i, j - 1), (i, j + 1)\}.$$

Кодирование состояния окрестности в 5-битное число:

$$\text{state} = f(A, B, C, D, E) = s(i, j) \times 16 + s(i, j - 1) \times 8 + s(i + 1, j) \times 4 + s(i, j + 1) \times 2 + s(i - 1, j) \times 1,$$

где функция f вычисляет численный код состояния окрестности.

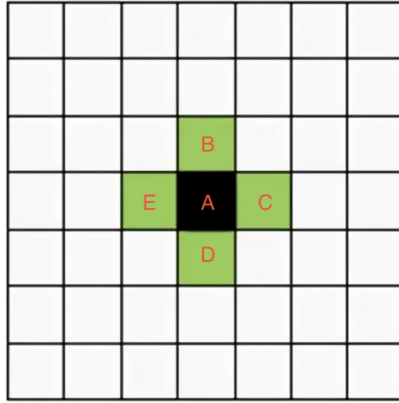


Рис. 1: Окрестность фон Неймана

1.3 Функция перехода

Правила перехода генерируются из числа 59350005, что эквивалентно $11100010100110010010110101_2$. Чтобы сформировать правила, необходимо это число в двоичной системе счисления дополнить нулями до 32 бит. Все переходы состояний представлены на Рис.2.

1.4 Граничные условия

В данной реализации использованы тороидальные (периодические) граничные условия, которые обеспечивают замкнутость пространства и исключают краевые эффекты. Математически это выражается через операцию взятия по модулю:

$$\text{cell}(x, y) = \text{cell}(x \bmod W, y \bmod H)$$

где:

- W — ширина поля

A	B	C	D	E	F
0	0	0	0	0	0
0	0	0	0	1	0
0	0	0	1	0	0
0	0	0	1	1	0
0	0	1	0	0	0
0	0	1	0	1	0
0	0	1	1	0	1
0	0	1	1	1	1
0	1	0	0	0	1
0	1	0	0	1	0
0	1	0	1	0	0
0	1	0	1	1	0
0	1	1	0	0	1
0	1	1	0	1	0
0	1	1	1	0	1
0	1	1	1	1	0
1	0	0	0	0	0
1	0	0	0	1	1
1	0	0	1	0	1
1	0	0	1	1	0
1	0	1	0	0	0
1	0	1	0	1	1
1	0	1	1	0	0
1	0	1	1	1	0
1	1	0	0	0	1
1	1	0	0	1	0
1	1	0	1	0	1
1	1	0	1	1	1
1	1	1	0	0	0
1	1	1	0	1	1
1	1	1	1	0	0
1	1	1	1	1	1

Рис. 2: Таблица истинности

- H — высота поля
- mod — операция взятия по модулю

Для клетки с координатами $(i, j) \in [0, W - 1] \times [0, H - 1]$ соседи определяются следующим образом:

$$B = \text{grid}[i, (j - 1) \bmod W]$$

$$C = \text{grid}[(i + 1) \bmod H, j]$$

$$D = \text{grid}[i, (j + 1) \bmod W]$$

$$E = \text{grid}[(i - 1) \bmod H, j]$$

2 Особенности реализации

2.1 class NeumannCellularAutomaton

Класс, содержащий все методы для работы с клеточным автоматом, включая инициализацию, обновление состояния и визуализацию.

```
variant = 39
day = 23
month = 3
year = 2005
n = 11 * variant * day * month * year

class NeumannCellularAutomaton:
    def __init__(self, width, height=None):
        self.width = width
        self.height = height if height else width
        self.grid = np.zeros((self.height, self.width), dtype=int)
        self.rules = self._generate_rules_from_id(n)

    def _generate_rules_from_id(self, student_id):

    def get_neumann_neighbors(self, x, y):

    def get_neighborhood_state(self, x, y):

    def update(self):

    def set_random_initial_state(self, density=0.5):

    def set_manual_initial_state(self, pattern):

    def visualize_simulation(self, iterations):

    def run_simulation(self, iterations, visualize=True):

    def run_console_simulation(self, iterations):
```

```

def visualize_simulation(self, iterations):
    def animate(frame):

def analyze_behavior(self, max_iterations=100):

def plot_behavior_analysis(self, live_history, density_history, max_iterations):

```

2.2 Функция main()

Функция отвечает за взаимодействие с пользователем. Реализована в виде цикла с меню выбора параметров.

На вход: параметры ширины поля, количества итераций и начальные условия от пользователя.

На выход: сигнал о завершении программы.

```

def main():
    print("Двумерный клеточный автомат с окрестностью фон Неймана")
    print("Функция: 59350005")

    try:
        width = int(input("Введите ширину поля: ") or 100)
        iterations = int(input("Введите количество итераций: ") or 100)
    except ValueError:
        print("Ошибка: введите целые числа")
        return

    automaton = NeumannCellularAutomaton(width)

    print("\nВыберите начальное состояние:")
    print("1 - Случайное распределение")
    print("2 - Волны")
    print("3 - Шахматная доска")
    print("4 - Спираль")
    print("5 - Сетка из точек")
    print("6 - Крест")

```

```

print("7 - Случайное плотное (70%)")
print("8 - Случайное разреженное (30%)")

choice = input("Ваш выбор (1-8): ") or "1"

patterns = {
    "1": "random",
    "2": "waves",
    "3": "checkerboard",
    "4": "spiral",
    "5": "dots_grid",
    "6": "cross",
    "7": "random_dense",
    "8": "random_sparse"
}

if choice in patterns:
    pattern = patterns[choice]
    if pattern.startswith("random"):
        density = 0.7 if pattern == "random_dense" else 0.3 if pattern == "random_sparse" else 0.5
        automaton.set_random_initial_state(density)
    else:
        automaton.set_manual_initial_state(pattern)
else:
    print("Неверный выбор, используется случайное распределение")
    automaton.set_random_initial_state(0.5)

display_mode = input("Режим отображения (1 - консоль, 2 - графика): ") or "2"

if display_mode == "1":
    automaton.run_simulation(iterations, visualize=False)
else:
    automaton.run_simulation(iterations, visualize=True)

analyze = input("Провести расширенный анализ поведения? (y/n): ")
if analyze.lower() == 'y':

```

```
automaton.analyze_behavior(iterations)
```

2.3 Генерация правил из номера (`_generate_rules_from_id`)

Функция преобразует число в набор правил для клеточного автомата. Номер переводится в двоичную систему, дополняется нулями до 32 бит, и каждый бит становится правилом для соответствующего состояния окрестности.

На вход: номер варианта *student_id*.

На выход: словарь правил *rules*.

```
def _generate_rules_from_id(self, student_id):
    binary_str = bin(student_id)[2:]
    while len(binary_str) < 32:
        binary_str = '0' + binary_str
    rules_binary = binary_str[-32:]
    rules = {}
    for i in range(32):
        rules[i] = int(rules_binary[i])
    return rules
```

2.4 Получение соседей по окрестности фон Неймана (`get_neumann_neighbors`)

Функция возвращает состояния четырех соседей клетки (x, y) с учетом тороидальных граничных условий. Используется операция взятия по модулю для обеспечения цикличности пространства.

На вход: координаты клетки (x, y) .

На выход: список состояний соседей $[, , ,]$.

```
def get_neumann_neighbors(self, x, y):
    neighbors = []
    neighbors.append(self.grid[(y - 1) % self.height, x]) # верхний
    neighbors.append(self.grid[y, (x + 1) % self.width]) # правый
    neighbors.append(self.grid[(y + 1) % self.height, x]) # нижний
    neighbors.append(self.grid[y, (x - 1) % self.width]) # левый
    return state
```

2.5 Кодирование состояния окрестности (get_neighborhood_state)

Функция вычисляет 5-битный код, представляющий состояние окрестности клетки. Каждому соседу соответствует свой вес в двоичном представлении.

На вход: координаты клетки (x, y) .

На выход: число от 0 до 31, кодирующее состояние окрестности.

```
def get_neighborhood_state(self, x, y):
    center = self.grid[y, x]
    neighbors = self.get_neumann_neighbors(x, y)

    state = center * 16 # центральная клетка (бит 4)
    state += neighbors[0] * 8 # верхний сосед (бит 3)
    state += neighbors[1] * 4 # правый сосед (бит 2)
    state += neighbors[2] * 2 # нижний сосед (бит 1)
    state += neighbors[3] * 1 # левый сосед (бит 0)
    return state
```

2.6 Обновление состояния автомата (update)

Функция выполняет один шаг эволюции клеточного автомата. Для каждой клетки вычисляется новое состояние на основе правил и текущей окрестности.

На вход: текущее состояние автомата.

На выход: обновленное состояние автомата.

```
def update(self):
    new_grid = np.zeros((self.height, self.width), dtype=int)
    for y in range(self.height):
        for x in range(self.width):
            neighborhood_state = self.get_neighborhood_state(x, y)
            new_grid[y, x] = self.rules[neighborhood_state]
    self.grid = new_grid
```

2.7 Установка случайного начального состояния (set_random_initial_state)

Функция инициализирует поле клеточного автомата случайными значениями с заданной плотностью.

На вход: параметр плотности `density`, по умолчанию 0.5.

На выход: инициализированная матрица состояний.

```
def set_random_initial_state(self, density=0.5):
    self.grid = np.random.choice([0, 1],
    size=(self.height, self.width),
    p=[1 - density, density])
```

2.8 Установка начального состояния (`set_manual_initial_state`)

Функция устанавливает начальную конфигурацию автомата согласно выбранному паттерну. Реализованы различные паттерны: случайное распределение, волны, шахматная доска, спираль и сетка из точек.

На вход: идентификатор паттерна *pattern*.

На выход: инициализированное начальное состояние.

```
def set_manual_initial_state(self, pattern):
    self.grid = np.zeros((self.height, self.width), dtype=int)
    center_x, center_y = self.width // 2, self.height // 2

    if pattern == "random":
        self.set_random_initial_state(0.5)

    elif pattern == "waves":
        for y in range(self.height):
            for x in range(self.width):
                if (y // 2) % 3 == 0:
                    self.grid[y, x] = 1

    elif pattern == "checkerboard":
        cell_size = 2
        for y in range(self.height):
            for x in range(self.width):
                if ((x // cell_size) + (y // cell_size)) % 2 == 0:
                    self.grid[y, x] = 1

    elif pattern == "spiral":
```

```

size = min(self.width, self.height) // 3
for i in range(size):
    angle = i * 0.3
    radius = i * 0.8
    x = int(center_x + radius * np.cos(angle))
    y = int(center_y + radius * np.sin(angle))
    if 0 <= x < self.width and 0 <= y < self.height:
        self.grid[y, x] = 1
        for dx in [-1, 0, 1]:
            for dy in [-1, 0, 1]:
                nx, ny = x + dx, y + dy
                if 0 <= nx < self.width and 0 <= ny < self.height and random.random() < 0.3:
                    self.grid[ny, nx] = 1

elif pattern == "dots_grid":
    spacing = 5
    for y in range(0, self.height, spacing):
        for x in range(0, self.width, spacing):
            if y < self.height and x < self.width:
                self.grid[y, x] = 1

elif pattern == "cross":
    self.grid[center_y, center_x] = 1
    self.grid[center_y - 1, center_x] = 1
    self.grid[center_y + 1, center_x] = 1
    self.grid[center_y, center_x - 1] = 1
    self.grid[center_y, center_x + 1] = 1

elif pattern == "random_dense":
    self.set_random_initial_state(0.7)
elif pattern == "random_sparse":
    self.set_random_initial_state(0.3)

def run_simulation(self, iterations, visualize=True):
    if visualize:
        self.visualize_simulation(iterations)

```

```

else:
self.run_console_simulation(iterations)

```

2.9 Консольная симуляция (run_console_simulation)

Функция выполняет симуляцию в консольном режиме с выводом статистики.

На вход: количество итераций iterations.

На выход: пошаговый вывод состояния в консоль.

```

def run_console_simulation(self, iterations):
print("Начальное состояние:")
self.print_grid()

print(f"\n{'Шаг':<6} {'Живые':<8} {'Плотность':<10}")
print("-" * 25)

for i in range(iterations + 1):
if i > 0:
self.update()

live_cells = np.sum(self.grid)
density = live_cells / (self.width * self.height)
print(f"{i:<6} {live_cells:<8} {density:.4f}")

if i < iterations:
print(f"\n|||ar {i + 1}:")
self.print_grid()
time.sleep(0.5)

```

2.10 Вывод сетки в консоль (print_grid)

Функция отображает текущее состояние автомата в консоли.

На вход: числовая матрица состояний.

На выход: последовательность строк в стандартный поток вывода.

```

def print_grid(self):
for row in self.grid:

```

```
print(''.join([' | ' if cell else ' ' for cell in row]))
```

2.11 Визуализация симуляции (visualize_simulation)

Функция создает анимацию эволюции клеточного автомата с одновременным отображением графика динамики количества "живых" клеток.

На вход: количество итераций *iterations*.

На выход: объект анимации библиотеки Matplotlib

```
def visualize_simulation(self, iterations):
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
    cmap = colors.ListedColormap(['white', 'black'])

    img = ax1.imshow(self.grid, cmap=cmap, interpolation='nearest')
    ax1.set_title(f'Клеточный автомат (шаг 0)')
    ax1.set_xticks([])
    ax1.set_yticks([])

    line, = ax2.plot([], [], 'b-', linewidth=2)
    ax2.set_xlabel('Шаг итерации')
    ax2.set_ylabel('Количество клеток в состоянии 1')
    ax2.set_title('Динамика состояния 1')
    ax2.grid(True, alpha=0.3)
    ax2.set_xlim(0, iterations)

    live_cells_history = [np.sum(self.grid)]

    def animate(frame):
        if frame > 0:
            self.update()
            current_live = np.sum(self.grid)
            live_cells_history.append(current_live)
        else:
            current_live = live_cells_history[0]

    img.set_array(self.grid)
```

```

ax1.set_title(f'Клеточный автомат (шаг {frame})')

x_data = list(range(frame + 1))
line.set_data(x_data, live_cells_history[:frame + 1])

if live_cells_history:
    max_live = max(live_cells_history)
    ax2.set_ylim(0, max(1, max_live * 1.1))

if hasattr(animate, 'annotation'):
    animate.annotation.remove()

if frame > 0:
    animate.annotation = ax2.annotate(f'{current_live}',
    xy=(frame, current_live),
    xytext=(5, 5), textcoords='offset points',
    bbox=dict(boxstyle='round,pad=0.3', facecolor='yellow', alpha=0.7))
    return img, line

anim = animation.FuncAnimation(
    fig, animate, frames=iterations + 1,
    interval=500, repeat=False, blit=False)

plt.tight_layout()
plt.show()
self.live_cells_history = live_cells_history

```

2.12 Анализ поведения (analyze_behavior)

Функция выполняет расширенный анализ поведения автомата, включая построение различных графиков и вычисление статистических характеристик.

На вход: максимальное число итераций *max_iterations*.

На выход: массивы данных о количестве живых клеток и плотности по итерациям.

```

def analyze_behavior(self, max_iterations=100):
    print("Расширенный анализ поведения автомата...")

```

```

initial_grid = self.grid.copy()
live_history = []
density_history = []

for i in range(max_iterations):
    live_cells = np.sum(self.grid)
    density = live_cells / (self.width * self.height)

    live_history.append(live_cells)
    density_history.append(density)
    self.update()

    if i > 0 and live_history[-1] == live_history[-2]:
        print(f"Стабилизация на шаге {i}: {live_cells} живых клеток")
        break

self.grid = initial_grid
self.plot_behavior_analysis(live_history, density_history, max_iterations)
return live_history, density_history

```

2.13 Построение графиков анализа (plot_behavior_analysis)

Функция создает комплексную визуализацию анализа поведения автомата.

На вход: массивы данных о количестве живых клеток и плотности по итерациям, максимальное число итераций.

На выход: объект фигуры библиотеки Matplotlib.

```

def plot_behavior_analysis(self, live_history, density_history, max_iterations):
    fig, ((ax1, ax2), (ax3, ax4)) = plt.subplots(2, 2, figsize=(12, 10))
    steps = list(range(len(live_history)))

    ax1.plot(steps, live_history, 'b-', linewidth=2, label='Живые клетки')
    ax1.set_xlabel('Шаг итерации')
    ax1.set_ylabel('Количество живых клеток')
    ax1.set_title('Динамика количества живых клеток')

```

```

ax1.grid(True, alpha=0.3)
ax1.legend()

ax2.plot(steps, density_history, 'r-', linewidth=2, label='Плотность')
ax2.set_xlabel('Шаг итерации')
ax2.set_ylabel('Плотность')
ax2.set_title('Динамика плотности')
ax2.grid(True, alpha=0.3)
ax2.legend()

ax3.hist(live_history, bins=20, alpha=0.7, color='green', edgecolor='black')
ax3.set_xlabel('Количество живых клеток')
ax3.set_ylabel('Частота')
ax3.set_title('Распределение количества живых клеток')
ax3.grid(True, alpha=0.3)

if len(live_history) > 1:
    changes = [live_history[i] - live_history[i-1]
    for i in range(1, len(live_history))]
    ax4.plot(steps[1:], changes, 'g-', linewidth=2, label='Изменение')
    ax4.axhline(y=0, color='red', linestyle='--', alpha=0.5)
    ax4.set_xlabel('Шаг итерации')
    ax4.set_ylabel('Изменение количества клеток')
    ax4.set_title('Скорость изменения')
    ax4.grid(True, alpha=0.3)
    ax4.legend()
plt.tight_layout()
plt.show()

print(f"\nСтатистика анализа:")
print(f"Начальное количество: {live_history[0]}")
print(f"Финальное количество: {live_history[-1]}")
print(f"Максимальное количество: {max(live_history)}")
print(f"Минимальное количество: {min(live_history)}")
print(f"Среднее количество: {np.mean(live_history):.2f}")

```

3 Результаты работы программы

Перед началом работы пользователь указывает размеры поля и количество итераций, также выбирает начальное состояние и выбирает режим отображения, как показано на Рис. 3 и Рис. 4.

```
59350005
Двумерный клеточный автомат с окрестностью фон Неймана
Функция: 59350005
Введите ширину поля: 30
Введите количество итераций: 10

Выберите начальное состояние:
1 - Случайное распределение
2 - Волны
3 - Шахматная доска
4 - Спираль
5 - Сетка из точек
6 - Крест
7 - Случайное плотное (70%)
8 - Случайное разреженное (30%)
Ваш выбор (1-8): 1
Режим отображения (1 - консоль, 2 - графика): 1
Начальное состояние:
[График начального состояния]
```

Шаг	Живые	Плотность
0	430	0.4778

Рис. 3: Валидация значений

```
Шаг 1:
[График 1-й итерации]
```

Шаг	Живые	Плотность
1	310	0.4960


```
Шаг 2:
[График 2-й итерации]
```

Шаг	Живые	Плотность
2	295	0.4720

Рис. 4: Первые итерации

Помимо консольного режима, автомат можно вывести в отдельное окно, как показано на рис.5-8, где представлены четыре различных начальных состояния клеточного автомата, при графическом выводе.



Рис. 5: (а) Спираль

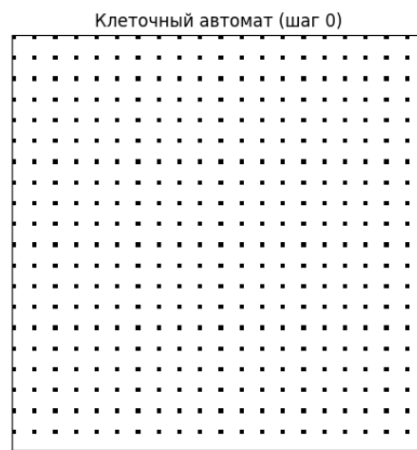


Рис. 6: (b) Сетка из точек

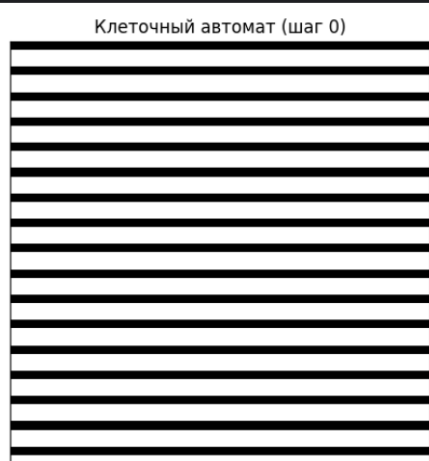


Рис. 7: (с) Волны (полосы)

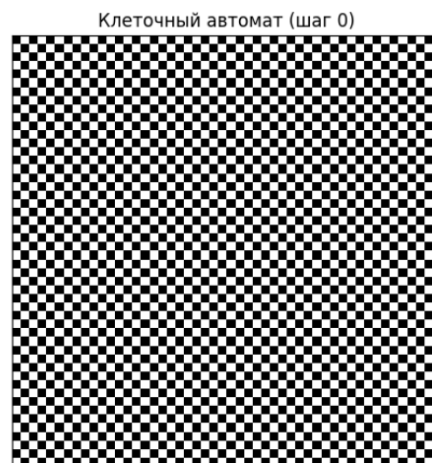


Рис. 8: (d) Шахматная доска

Ниже приведен результат работы клеточного автомата в графическом режиме на Рис. 10 и расширенный анализ, который показывает всю статистику на момент работы автомата Рис. 9.

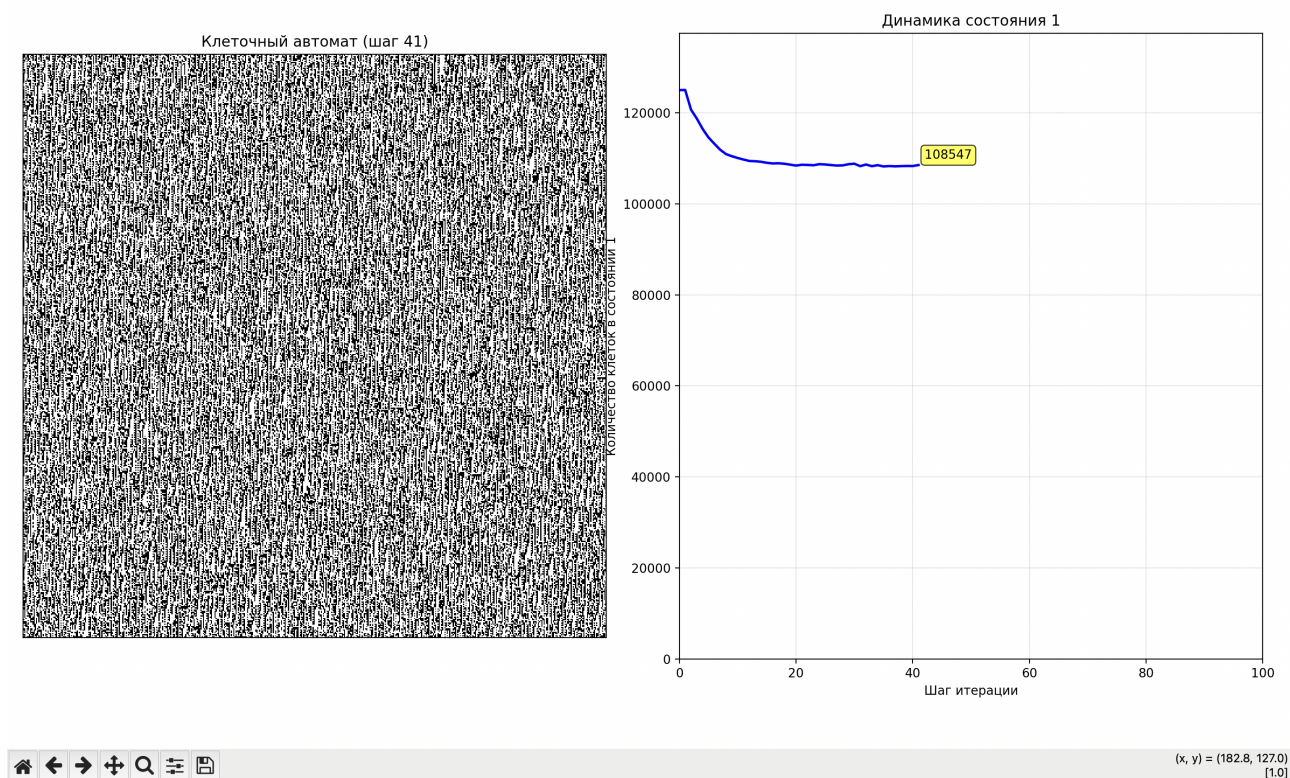


Рис. 9: Режим работы автомата

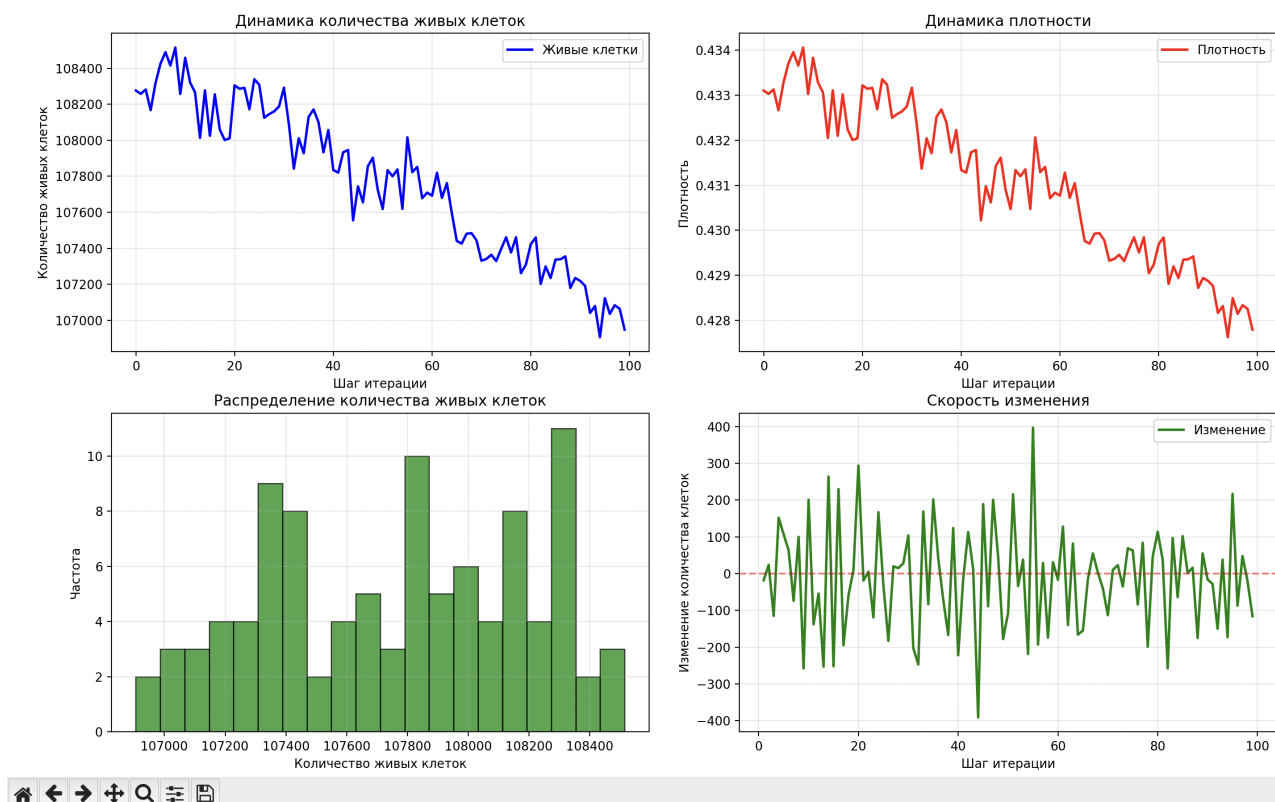


Рис. 10: Режим аналитики

4 Анализ клеточного автомата

Определение Класса IV по классификации Вольфрама – результатом эволюции являются структуры, которые взаимодействуют сложным образом с формированием локальных, устойчивых структур. В результате эволюции могут получаться некоторые последовательности. Локальные изменения в начальных условиях оказывают неопределяемое влияние на ход эволюции системы.

Исследованный клеточный автомат демонстрирует поведение, характерное для Класса IV по классификации Вольфрама. Это подтверждается несколькими ключевыми наблюдениями:

- Автомат стремится к устойчивому значению
- От незначительных изменений начальных условий поведение автомата не меняется
- Система способна поддерживать паттерны.

При начальных условиях шахматной доски, сетки или креста автомат приходит в статичное состояние образуя паттерны или группу сменных паттерна, которые показаны на Рис. 11 - 13. Крест приходит в статичное состояние спустя 274 итераций. Сетка не меняет свой рисунок, однако система динамически перемещается сверху вниз. Шахматная доска чередует 12 постоянных паттернов, которые циклично заменяют друг друга.

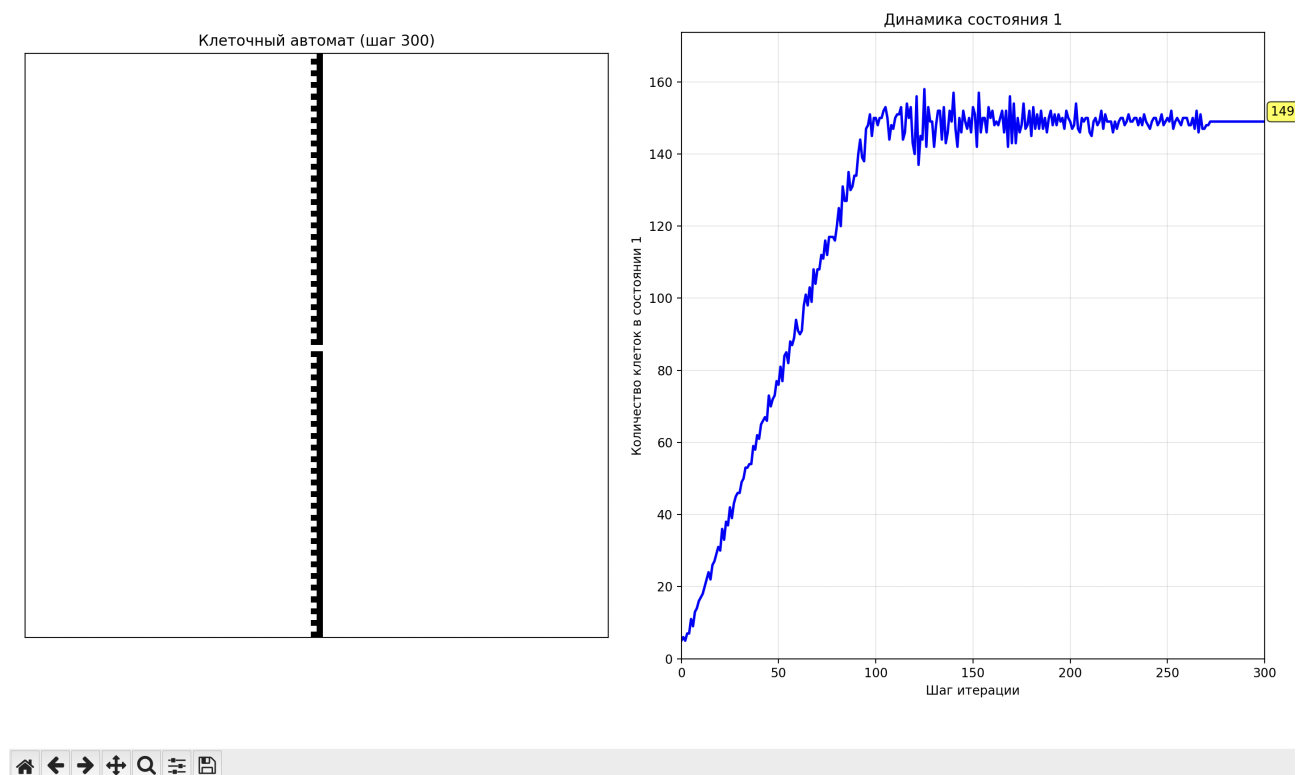


Рис. 11: Крест

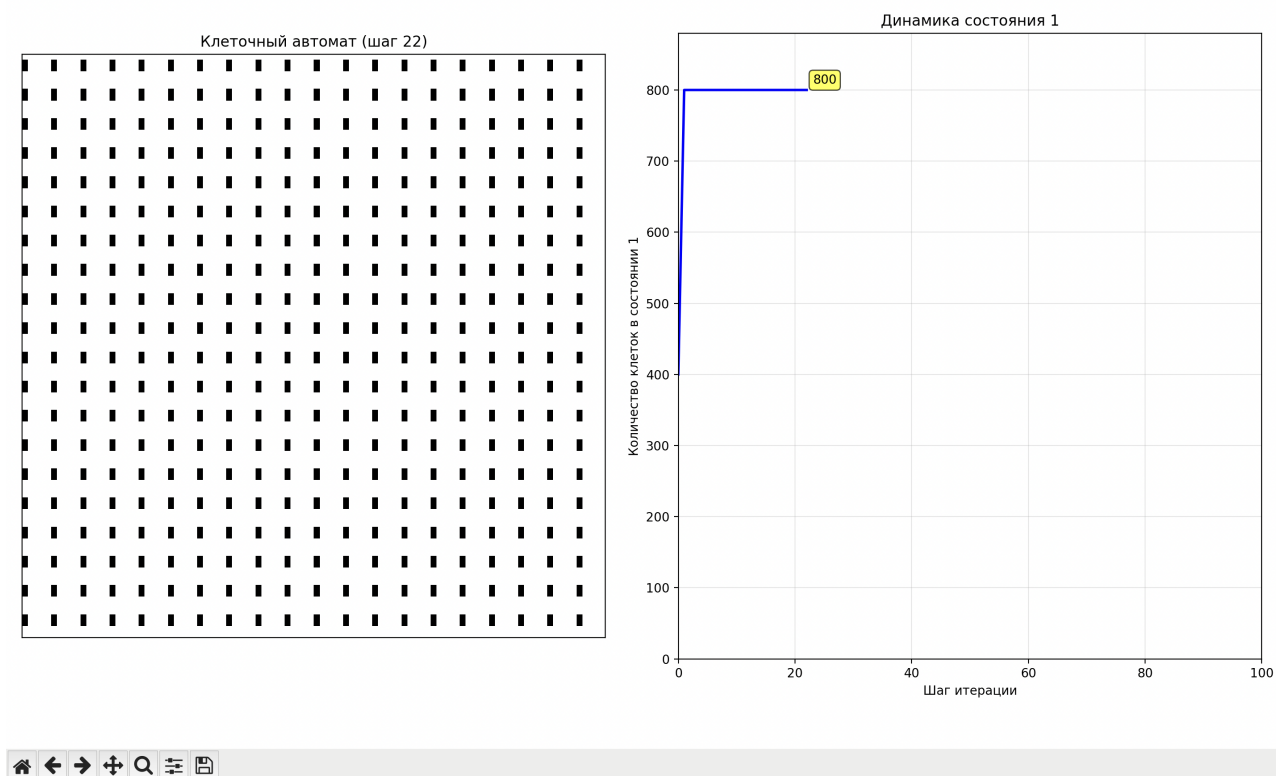


Рис. 12: Сетка

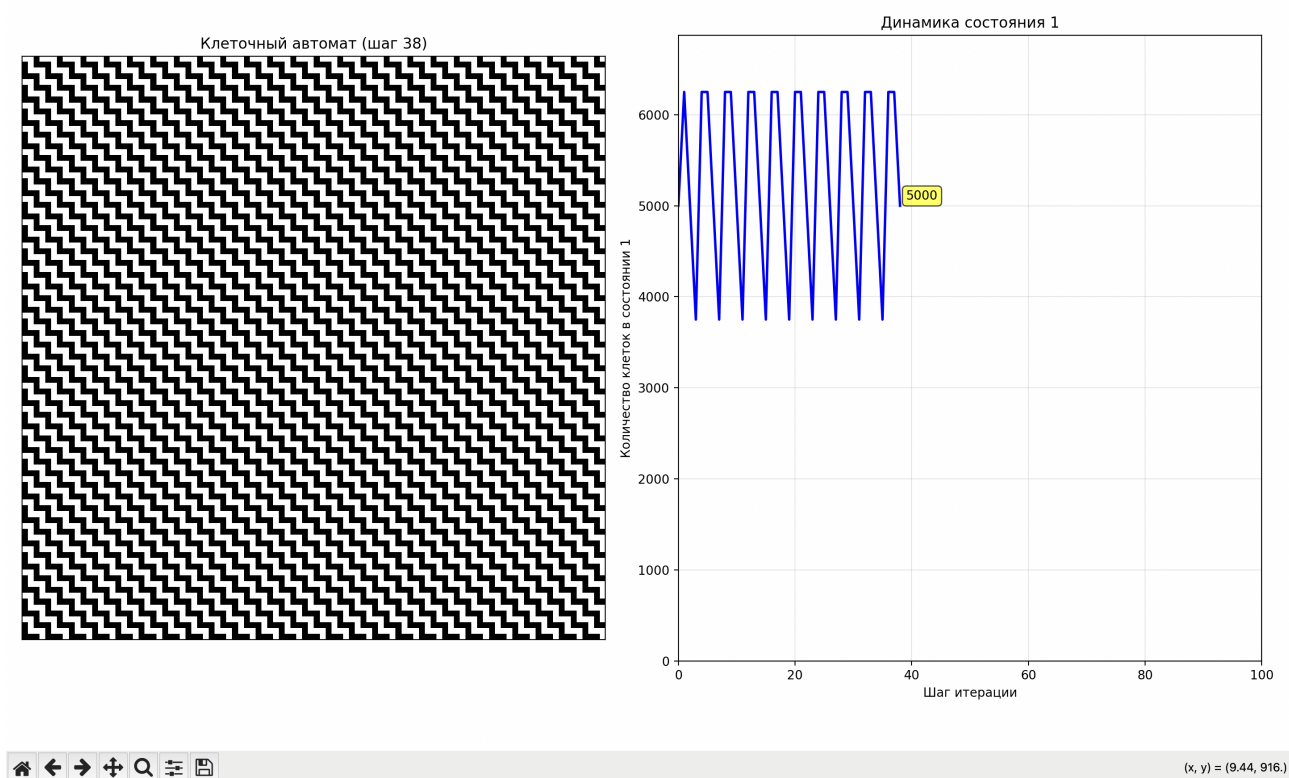


Рис. 13: Шахматная доска

Наиболее яркой характеристикой является стабилизация плотности живых клеток на уровне 43%, наблюдаемая практически при всех начальных условиях. Эта величина не зависит от размера поля и количества итераций.

Для исходного варианта 59350005 выявлено 32 правила:

- Правил с результатом 1: 13 (40.6%)
- Правил с результатом 0: 19 (59.4%)

Анализ возникающих паттернов показывает отсутствие стабильных структур типа осцилляторов или глайдеров. Все образующиеся конфигурации носят временный характер и быстро разрушаются под влиянием окружающего "шума". Характерным примером такого поведения является Спираль, поскольку система разрастается сохраняя определенный рисунок и правила распространения, однако в конечном итоге система превращается в полностью хаотичную и паттерны исчезают.

Заключение

В данной работе был реализован двумерный клеточный автомат с окрестностью Фон-Неймана. В выбранной модели каждая ячейка взаимодействует с пятью элементами: самой собой и четырьмя соседями по вертикали и горизонтали. Функция перехода задана числом 59350005.

Размер поля вводится пользователем, что обеспечивает возможность адаптации модели под различные задачи. Начальные условия могут быть сгенерированы случайно или использовать определенные рисунки. Торроидальные (циклические) граничные условия означают, что противоположные края поля связаны между собой, формируя замкнутую поверхность. Это исключает влияние краевых эффектов и позволяет исследовать внутреннюю динамику автомата без искажений, связанных с границами.

В рамках программы была реализована дополнительная функция — расширенный анализ автомата.

Достоинства реализации:

- Инкапсуляция - все данные и методы работы с автоматом содержатся в одном классе
- Программа не зависит от входной функции
- Вместе с автоматом выводится динамический график, отображающий состояние автомата в данный момент
- Работоспособность при больших значениях (программа работает в штатном режиме при значениях поля 1000 x 1000).

Недостатки реализации:

- Статические правила - невозможность изменения правил во время выполнения
- Потребление памяти - хранение полной истории для анимации
- Настройки работы автомата задаются программно и пользователь не имеет возможности выставить скорость работы, пройти по шагам или остановить, а после воспроизвести работу автомата.

Масштабирование:

- Динамическое изменение настроек во время выполнения
- Возможность задания 2 сторон поля, отображения на неквадратном поле.

Список материалов

- [1] «New kind of science», S. Wolfram. — Текст: электронный — URL: [https://web.archive.org/web/20070508234748 http://www.wolframscience.com/](https://web.archive.org/web/20070508234748/http://www.wolframscience.com/) (дата обращения: 22.10.2025).
- [2] Секция «Телематика». — Текст: электронный — URL: <https://tema.spbstu.ru/tgraph/> (дата обращения: 22.10.2025).